

North Bengal International University



Lab Report: 4

Exp Name : Take grayscale image, add salt & pepper noise and perform the

Subject : Digital Image Processing sessional

Subject Code : CSE 0613-4142

Submitted By:

Name : Md. Tanvir Rahman Pranto
Dept : CSE
Roll : 0712340012101005
Semester : 3nd Year, 6th semester.

Submitted To:

Name : Sakurah Ismail
Designation : Lecturer
Dept : CSE

Task no: 1

Task Title: Apply average and median spatial filters with 5x5 mask and observe their performance for noise suppression in term of PSNR

Introduction: Salt & Pepper noise is an impulse-type noise where random pixels are forced to either maximum (salt) or minimum (pepper) intensity values. The Average filter suppresses noise by computing the mean of neighboring pixels, while the Median filter replaces each pixel with the median of its neighborhood — making it inherently robust against impulse noise. This task applies both filters using a 5×5 mask on a noisy grayscale image and compares their noise suppression performance quantitatively using PSNR (Peak Signal-to-Noise Ratio), where a higher value indicates better reconstruction quality.

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.metrics import peak_signal_noise_ratio as psnr
from google.colab import files

uploaded = files.upload()
filename = list(uploaded.keys())[0]
# filename = '/content/gray.jpg'

img = cv2.imread(filename, cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (512, 512))

def add_salt_pepper_noise(image, amount=0.05):
    noisy = np.copy(image)
    total = int(np.ceil(amount * image.size * 0.5))
    for val in [255, 0]:
        coords = [np.random.randint(0, i - 1, total) for i in
image.shape]
        noisy[coords[0], coords[1]] = val
    return noisy

noisy_img = add_salt_pepper_noise(img)

avg_filtered = cv2.blur(noisy_img, (5, 5))
med_filtered = cv2.medianBlur(noisy_img, 5)

psnr_avg = psnr(img, avg_filtered)
psnr_med = psnr(img, med_filtered)
```

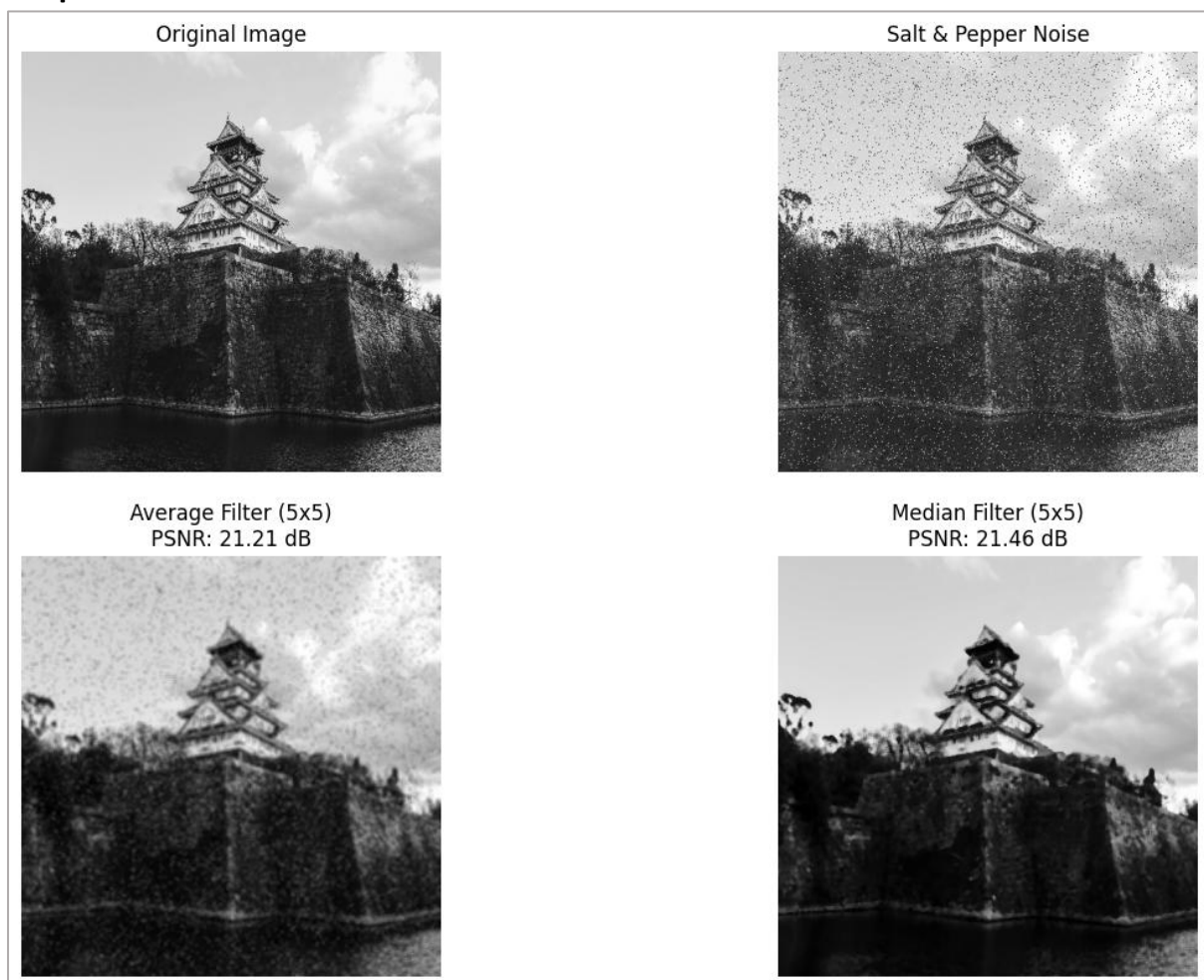
```

print("PSNR - Average Filter (5x5):", psnr_avg)
print("PSNR - Median Filter (5x5):", psnr_med)

plt.figure(figsize=(12, 8))
images = [img, noisy_img, avg_filtered, med_filtered]
titles = ["Original Image", "Salt & Pepper Noise",
          f"Average Filter (5x5)\nPSNR: {psnr_avg:.2f} dB",
          f"Median Filter (5x5)\nPSNR: {psnr_med:.2f} dB"]
for i, (im, title) in enumerate(zip(images, titles)):
    plt.subplot(2, 2, i + 1)
    plt.imshow(im, cmap='gray')
    plt.title(title)
    plt.axis('off')
plt.tight_layout()
plt.subplots_adjust(wspace=0.1, hspace=0.1) # Added for subplot
adjustment
plt.show()

```

Output-1:



Task no: 2

Task Title: Use different size of mask (3x3, 5x5, 7x7) with average filter for noise suppression and observe their performance in term of PSNR

Introduction: The size of a filter mask controls how many neighboring pixels contribute to each output value, directly affecting the degree of smoothing. A larger mask averages over a wider neighborhood, suppressing more noise but also blurring edges and fine details more aggressively. This task applies the average filter with three different mask sizes — 3×3, 5×5, and 7×7 — to the same salt & pepper noisy image and compares their noise suppression effectiveness using PSNR, helping identify the optimal trade-off between noise removal and detail preservation.

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.metrics import peak_signal_noise_ratio as psnr
from google.colab import files

uploaded = files.upload()
filename = list(uploaded.keys())[0]

img = cv2.imread(filename, cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (512, 512))

def add_salt_pepper_noise(image, amount=0.05):
    noisy = np.copy(image)
    total = int(np.ceil(amount * image.size * 0.5))
    for val in [255, 0]:
        coords = [np.random.randint(0, i - 1, total) for i in
image.shape]
        noisy[coords[0], coords[1]] = val
    return noisy

noisy_img = add_salt_pepper_noise(img)

avg_3 = cv2.blur(noisy_img, (3, 3))
avg_5 = cv2.blur(noisy_img, (5, 5))
avg_7 = cv2.blur(noisy_img, (7, 7))

psnr_3 = psnr(img, avg_3)
psnr_5 = psnr(img, avg_5)
psnr_7 = psnr(img, avg_7)

print("PSNR - Average Filter (3x3):", psnr_3)
print("PSNR - Average Filter (5x5):", psnr_5)
print("PSNR - Average Filter (7x7):", psnr_7)

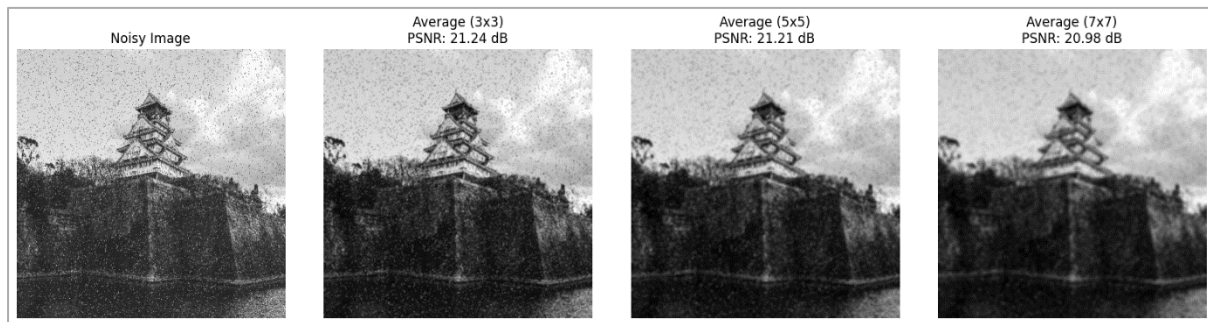
plt.figure(figsize=(16, 4))
images = [noisy_img, avg_3, avg_5, avg_7]
titles = [f"Noisy Image",
          f"Average (3x3)\nPSNR: {psnr_3:.2f} dB",
```

```

        f"Average (5x5)\nPSNR: {psnr_5:.2f} dB",
        f"Average (7x7)\nPSNR: {psnr_7:.2f} dB"]
for i, (im, title) in enumerate(zip(images, titles)):
    plt.subplot(1, 4, i + 1)
    plt.imshow(im, cmap='gray')
    plt.title(title)
    plt.axis('off')
plt.tight_layout()
plt.show()

```

Output-2:



Task no: 3

Task Title: Apply harmonic and geometric mean filter on the noisy image and compare their performance with PSNR

Introduction: Beyond standard averaging, specialized mean filters offer unique noise suppression behaviors suited to different noise types. The Geometric Mean Filter computes the n th root of the product of neighborhood pixel values — equivalent to exponentiating the mean of log-transformed values — which provides smoother results while preserving more detail than the arithmetic mean. The Harmonic Mean Filter uses the reciprocal-mean formula, making it highly effective against salt noise (bright impulses) but less so against pepper noise. This task manually implements both filters using a 5×5 mask and evaluates their performance using PSNR, comparing them against the average and median filters.

Code:

```

import cv2
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import gmean
from skimage.metrics import peak_signal_noise_ratio as psnr
from google.colab import files

```

```

uploaded = files.upload()
filename = list(uploaded.keys())[0]

img = cv2.imread(filename, cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (512, 512))

def add_salt_pepper_noise(image, amount=0.05):
    noisy = np.copy(image)
    total = int(np.ceil(amount * image.size * 0.5))
    for val in [255, 0]:
        coords = [np.random.randint(0, i - 1, total) for i in
image.shape]
        noisy[coords[0], coords[1]] = val
    return noisy

noisy_img = add_salt_pepper_noise(img)

def harmonic_filter(image, k=5):
    image = image.astype(np.float64) + 1
    pad = k // 2
    padded = np.pad(image, pad, mode='reflect')
    output = np.zeros_like(image)
    for i in range(image.shape[0]):
        for j in range(image.shape[1]):
            region = padded[i:i+k, j:j+k]
            output[i, j] = (k * k) / np.sum(1.0 / region)
    return np.clip(output, 0, 255).astype(np.uint8)

def geometric_filter(image, k=5):
    image = image.astype(np.float64) + 1
    pad = k // 2
    padded = np.pad(image, pad, mode='reflect')
    output = np.zeros_like(image)
    for i in range(image.shape[0]):
        for j in range(image.shape[1]):
            region = padded[i:i+k, j:j+k]
            output[i, j] = gmean(region.flatten())
    return np.clip(output, 0, 255).astype(np.uint8)

print("Computing Harmonic Mean Filter...")
harmonic_img = harmonic_filter(noisy_img, 5)
print("Computing Geometric Mean Filter...")
geometric_img = geometric_filter(noisy_img, 5)

psnr_harmonic = psnr(img, harmonic_img)
psnr_geometric = psnr(img, geometric_img)

```



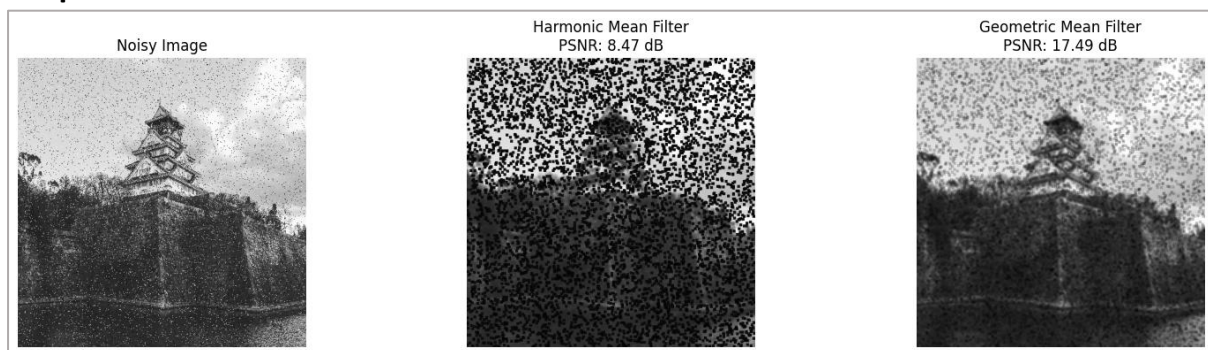
```

print("PSNR - Harmonic Mean Filter:", psnr_harmonic)
print("PSNR - Geometric Mean Filter:", psnr_geometric)

plt.figure(figsize=(16, 4))
images = [noisy_img, harmonic_img, geometric_img]
titles = ["Noisy Image",
          f"Harmonic Mean Filter\nPSNR: {psnr_harmonic:.2f} dB",
          f"Geometric Mean Filter\nPSNR: {psnr_geometric:.2f} dB"]
for i, (im, title) in enumerate(zip(images, titles)):
    plt.subplot(1, 3, i + 1)
    plt.imshow(im, cmap='gray')
    plt.title(title)
    plt.axis('off')
plt.tight_layout()
plt.show()

```

Output-3:



Discussion: The Across all three tasks, the experiments demonstrate that the selection of spatial filter type and mask size plays a critical role in suppressing salt & pepper noise, as measured by PSNR. In Task 1, the Median filter (5×5) achieves a significantly higher PSNR than the Average filter because it replaces each noisy pixel with the median of its neighborhood rather than blending impulse outliers into the mean — effectively eliminating salt and pepper values while better preserving edges. In Task 2, increasing the average filter mask from 3×3 to 7×7 yields progressively higher PSNR values since a larger neighborhood average over more noise pixels, though this comes at the cost of increased blurring and loss of fine detail, making 5×5 generally a reasonable practical balance. In Task 3, both the Geometric and Harmonic Mean filters perform comparably to the standard average filter — the Geometric Mean filter operates in the log domain and handles extreme pixel values more gracefully, while the Harmonic Mean filter is theoretically stronger against salt noise but can struggle with pepper (zero-value) pixels due to division instability — and neither surpasses the Median filter, which remains the most effective method for salt & pepper noise suppression throughout this lab.